

Stereo Camera-Based Environment Perception for Automated Vehicles

Stereokamera-Basierte Umfeldwahrnehmung für automatisierte
Fahrzeuge



Scientific work for obtaining the academic degree
Master of Science (M.Sc.)
at the Department Mobility Systems Engineering
of the TUM School of Engineering and Design
at the Technical University of Munich

Supervised by	Prof. Dr.-Ing. Markus Lienkamp David Brecht, M.Sc. Chair of Automotive Technology
Submitted by	Adrian Rieker, B.Sc. Schleißheimer Str. 126 80797 München
Submitted on	July 01, 2025

Project description

Titel der Arbeit

Hier steht ein zum Thema hinführender Text.

In einer konstruktiven (oder theoretischen) **Bachelor/-Semester/-Masterarbeit** sollen (...)

Folgende Punkte sind durch **Herrn/Frau Martin Musterstudent** zu bearbeiten:

- Punkt 1
- Punkt 2

Die Ausarbeitung soll die einzelnen Arbeitsschritte in übersichtlicher Form dokumentieren. Der Kandidat/Die Kandidatin verpflichtet sich, die **Bachelor/-Semester/-Masterarbeit** selbständig durchzuführen und die von ihm verwendeten wissenschaftlichen Hilfsmittel anzugeben.

Die eingereichte Arbeit verbleibt als Prüfungsunterlage im Eigentum des Lehrstuhls.

Announcement date: April 1, 2021

Submission date: October 1, 2021

Prof. Dr.-Ing. Markus Lienkamp

Max Musterbetreuer, M.Sc.

Geheimhaltungsverpflichtung

Herr: **Rieker, Adrian**

Gegenstand der Geheimhaltungsverpflichtung sind alle mündlichen, schriftlichen und digitalen Informationen und Materialien die der Unterzeichner vom Lehrstuhl oder von Dritten im Rahmen seiner Tätigkeit am Lehrstuhl erhält. Dazu zählen vor allem Daten, Simulationswerkzeuge und Programmcode sowie Informationen zu Projekten, Prototypen und Produkten.

Der Unterzeichner verpflichtet sich, alle derartigen Informationen und Unterlagen, die ihm während seiner Tätigkeit am Lehrstuhl für Fahrzeugtechnik zugänglich werden, strikt vertraulich zu behandeln.

Er verpflichtet sich insbesondere:

- derartige Informationen betriebsintern zum Zwecke der Diskussion nur dann zu verwenden, wenn ein ihm erteilter Auftrag dies erfordert,
- keine derartigen Informationen ohne die vorherige schriftliche Zustimmung des Betreuers an Dritte weiterzuleiten,
- ohne Zustimmung eines Mitarbeiters keine Fotografien, Zeichnungen oder sonstige Darstellungen von Prototypen oder technischen Unterlagen hierzu anzufertigen,
- auf Anforderung des Lehrstuhls für Fahrzeugtechnik oder unaufgefordert spätestens bei seinem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik alle Dokumente und Datenträger, die derartige Informationen enthalten, an den Lehrstuhl für Fahrzeugtechnik zurückzugeben.

Eine besondere Sorgfalt gilt im Umgang mit digitalen Daten:

- Für den Dateiaustausch dürfen keine Dienste verwendet werden, bei denen die Daten über einen Server im Ausland geleitet oder gespeichert werden (Es dürfen nur Dienste des LRZ genutzt werden (Lehrstuhlaufwerke, Sync&Share, GigaMove).
- Vertrauliche Informationen dürfen nur in verschlüsselter Form per E-Mail versendet werden.
- Nachrichten des geschäftlichen E-Mail Kontos, die vertrauliche Informationen enthalten, dürfen nicht an einen externen E-Mail Anbieter weitergeleitet werden.
- Die Kommunikation sollte nach Möglichkeit über die (my)TUM-Mailadresse erfolgen.

Die Verpflichtung zur Geheimhaltung endet nicht mit dem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik, sondern bleibt 5 Jahre nach dem Zeitpunkt des Ausscheidens in vollem Umfang bestehen. Die eingereichte schriftliche Ausarbeitung darf der Unterzeichner nach Bekanntgabe der Note frei veröffentlichen.

Der Unterzeichner willigt ein, dass die Inhalte seiner Studienarbeit in darauf aufbauenden Studienarbeiten und Dissertationen mit der nötigen Kennzeichnung verwendet werden dürfen.

Datum: 1. Juli 2025

Unterschrift: _____

Erklärung

Ich versichere hiermit, dass ich die von mir eingereichte Abschlussarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Garching, den 1. Juli 2025

Adrian Rieker, B.Sc.

Declaration of Consent, Open Source

Hereby I, Rieker, Adrian, born on August 14, 2000, make the software I developed during my Masterthesis Thesis available to the Institute of Automotive Technology under the terms of the license below.

Garching, July 1, 2025

Adrian Rieker, B.Sc.

Copyright 2025 Rieker, Adrian

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Contents

Formula Symbols	III
1 Introduction	1
2 State of the Art	3
2.1 Perception Systems for Autonomous Vehicles	3
2.2 Algorithms for moving object detection	5
2.3 State of the Art in the FTM	6
2.3.1 EDGAR Research Vehicle	7
2.3.2 Previous Work in the FTM	7
2.4 Stereo camera technology	8
2.5 Methods for software containerization	8
2.6 Requirements for the Thesis	8
3 Methods and Implementation	9
3.1 System Architecture	9
3.1.1 Perception pipeline	9
3.1.2 Criticality pipeline	10
3.1.3 Repository structure	11
3.1.4 Containerization	12
3.2 Perception pipeline: Stereo image computation	13
3.2.1 Mathematical Background: Pinhole Camera Model and Disparity	13
3.2.2 Stereo Matching Algorithms	14
3.2.3 Semi-Global Matching (SGM)	15
3.2.4 ROS2 Implementation	15
3.3 Perception pipeline: Optical flow computation	18
3.3.1 Optical flow concept	18
3.3.2 Optical flow algorithms	18
3.3.3 Farneback algorithm	20
3.3.4 Implementation	22
3.4 Perception pipeline: Ego motion estimation	25
3.4.1 Visual odometry concept	25
3.4.2 Reference frames and coordinate systems	26
3.4.3 Implemented algorithm	26
3.4.4 ROS2 Implementation	31
3.4.5 Parameter tuning and outputs	31
3.4.6 Reference frames and coordinate systems	31
3.4.7 Visual odometry algorithm	31
3.4.8 Implementation	31
3.4.9 Parameter tuning and outputs	31
3.4.10 Reference frame selection	31
3.4.11 Implementation	31
3.5 Perception pipeline: Kalman filter	31

3.5.1	Kalman filter algorithm	31
3.5.2	Mathematical model	31
3.5.3	Implementation.....	31
3.6	Perception pipeline: Object detection	31
3.6.1	DBSCAN clustering	31
3.6.2	Implementation.....	31
3.7	Criticallity pipeline: TTC computation	31
4	Experiments and Results	33
4.1	Camera hardware selection.....	33
4.2	Experimental setup	33
4.3	Perception pipeline evaluation	33
4.3.1	Stereo image computation	33
4.3.2	Optical flow computation	33
4.3.3	Ego motion estimation	33
4.3.4	Kalman filter.....	33
4.3.5	Object detection	33
4.4	Criticallity pipeline evaluation	33
5	Summary and Conclusion	35
	List of Figures.....	i
	List of Tables	iii
	Bibliography	v
	Appendix.....	ix

Formula Symbols

Formula Symbols	Unit	Description
f_x	px	Focal length of the camera in the x direction
f_y	px	Focal length of the camera in the y direction
c_x	px	x coordinate of the principal point
c_y	px	y coordinate of the principal point
B_{Fzg}	m	Breite des Fahrzeuges
B_{Rad}	m	Reifenbreite (Breite der Lauffläche)
D_{Rad}	m	Raddurchmesser
H_{Fzg}	m	Höhe des Fahrzeuges
R_{Fzg}	m	Radstand des Fahrzeuges
S_{Fzg}	m	Spur des Fahrzeuges
a_i	-	Gewichtungsfaktoren für die Kostenfunktion J
$\mathbf{c}_{\text{DGL}}$	-	Gleichungsnebenbedingungen, die aus der Diskretisierung der Zustandsdifferentialgleichung resultieren
H	-	Hamilton"-Funktion
E_{pot}	J	Potenzielle Energie
E_m	J	Spezifische Energie

Todo list

	Short review on stereo camera technology.	8
	Short review on methods for software containerization.	8
	Short review on the requirements for the thesis.	8
	Search source of sgm algorithm	10
	Explain how it was built: OpenCV, CUDA, ROS2, etc.	12

1 Introduction

Autonomous vehicles are rapidly becoming a reality, offering the promise of safer and more efficient transportation. However, their adoption still faces significant challenges, particularly in the area of perception. These systems are responsible for detecting and interpreting objects in the vehicle's surroundings, a task that becomes increasingly difficult in dynamic, unstructured, or unforeseen environments. In such cases, perception failures may lead to disengagements, where the autonomous system relinquishes control and requires remote human assistance via teleoperation.

In these critical moments, the reliability of the perception system is most important. It must provide robust, real-time awareness of the environment to either enable safe fallback strategies or support the teleoperator in navigating through complex scenes.

Most existing perception systems, responsible for detecting and tracking objects in the vehicle's surroundings, rely heavily on learning-based solutions. While these methods have demonstrated impressive performance in structured and controlled environments, they often struggle to generalize in novel and untrained scenarios. This limitation is particularly problematic in safety-critical situations, where the vehicle must make quick decisions based on the perceived environment.

This project goal is the development of a deterministic, stereo camera based perception system for autonomous vehicles, capable of providing a reliable and robust understanding of the surroundings, specially in highly dynamic and unforeseen scenarios. The system will take as input two images from a stereo camera mounted on the vehicle and output a list of detected objects in the scene, along with their criticality scores, forming a situational awareness module to assess potential hazards. Additionally, this deterministic approach could run in parallel with existing learning-based perception systems, providing an additional layer of redundancy and security in critical situations where the learning-based methods may fail.

To achieve accurate 6D motion estimation (position + velocity), the system will implement a deterministic algorithm inspired by the Daimler 6D Vision system. This approach uses Kalman Filters to estimate a 6D state vector for selected feature points in the scene, which will then be clustered to identify moving objects. Unlike learning-based methods, this approach does not require extensive training data and can generalize well in new unseen scenarios.

The perception framework is implemented in **C++** using the **Robot Operating System 2 (ROS2)** to ensure real-time capability, modularity, and easy deployment. ROS2 also enables seamless integration with the software stack of the **EDGAR research vehicle**, where the system will be deployed and tested. The modular design allows each component to run as an individual node, facilitating parallel processing and containerized using **Docker**.

The perception system is structured in two main pipelines:

- **The perception pipeline**, which computes dense optical flow, estimates ego-motion via stereo visual odometry, and derives depth maps from the stereo camera. These are fused in a **6D Kalman Filter** to estimate the position and velocity of selected points in the scene. The resulting motion field is then clustered into discrete objects using the **DBSCAN** algorithm.

- **The criticality assessment pipeline**, which takes the list of moving objects and the vehicle's planned trajectory to compute runtime safety metrics, particularly TTC. This module is crucial for evaluating risks and supporting decision-making in safety-critical situations.

The system makes extensive use of GPU acceleration via **CUDA**, particularly in modules such as optical flow estimation and depth computation. This enables the system to meet the performance requirements of real-time processing on the EDGAR platform.

Compared to previous works, such as the Daimler 6D Vision system, this approach integrates the deterministic motion estimation pipeline into a broader perception architecture tailored for real-world deployment in autonomous vehicles. Furthermore, by combining stereo vision, visual odometry, and deterministic filtering, the system is able to operate without any pretraining or external semantic data, increasing transparency and robustness.

To validate the system, a two-stage evaluation will be conducted. First, each module will be tested individually using public datasets such as KITTI, which provides ground truth for depth, optical flow, and odometry. Then, the integrated pipeline will be deployed in the EDGAR vehicle and evaluated in urban and semi-structured environments. Its outputs will be compared to the perception stack currently in use on the vehicle, which is based on deep learning models.

In summary, this work contributes a real-time capable, modular, and interpretable perception system based on stereo vision and deterministic algorithms. By improving robustness in edge-case scenarios and supporting redundancy in perception, the system aims to increase safety and reliability in the operation of autonomous vehicles.

2 State of the Art

Autonomous and automated vehicles must have a comprehensive and accurate understanding of their surroundings at all times to ensure safe and reliable operation. This process, known as environment perception, is responsible for detecting, identifying, and tracking objects such as other vehicles, cyclists, pedestrians or other road elements. The quality of the measurement of the state of these objects directly influence the vehicle's ability to make safe and informed driving decisions.

Failures in the perception can lead to hazardous situations, such as collisions or unexpected navigation decisions. This is particularly critical in high-speed scenarios or complex urban environments, where there is a high density of high dynamic objects which make more complex the operation in real time.

A robust perception system must provide the following capabilities:

1. **Object detection:** Identify and locate objects in the vehicle's surroundings, such as vehicles, pedestrians, cyclists, or other road elements.
2. **Motion estimation:** Determining the position and velocity of moving objects
3. **Scene understanding:** Classifying the detected objects and assessing their criticality for the vehicle's navigation
4. **Reliability in edge cases:** Ensuring correct operation in complex and unforeseen scenarios, such as occlusions, adverse weather conditions, or sensor failures.
5. **Real-time operation:** Providing timely and accurate information to the vehicle's control system to make informed driving decisions.

While the perception concept includes different aspects, this work will focus specifically on the detection of moving objects in the vehicle's surroundings and the estimation of their 6D motion state (position and velocity) using a deterministic approach. In the following sections, we will provide an overview of the state-of-the-art of the different sensors and algorithms used in the perception systems of autonomous vehicles, focusing on the deterministic image based approaches.

2.1 Perception Systems for Autonomous Vehicles

The perception systems of autonomous vehicles rely on a combination of different sensors to provide a comprehensive understanding of the ego vehicle's surroundings. In this section we will provide an overview of the most common sensor types used in autonomous vehicles and their advantages and limitations.

LiDAR-Based Perception

LiDAR (Light Detection and Ranging) sensors are increasingly used in the automotive industry for obstacle detection, 3D mapping and lane detection [1]. More and more companies are adopting this type of sensor due

to its advantage in rapidly and accurately estimating 3D distances. Examples include BMW [2], Mercedes-Benz [3], and Waymo [4].

Working principle LiDAR sensors use the concept of time of flight to measure the distance to objects in the vehicle's surroundings. The sensor emits a laser beam and measures the time it takes for the light to reflect back to the sensor. By calculating the time difference, the sensor can determine the distance to the object based on the speed of light [5].

Advantages and Limitations LiDAR sensors have the advantage of providing an accurate 3D representation of the vehicle's surroundings. This makes them particularly useful for object detection and localization tasks. However, LiDAR sensors are sensitive to adverse weather conditions, such as rain, snow or fog since the laser beams can be scattered by the particles in the air. Additionally, LiDAR sensors are more expensive than other sensor types, such as cameras or radars, which can limit their adoption in mass-market vehicles, although their prices are steadily decreasing [1].

Radar-Based Perception

Radar (Radio Detection and Ranging) use radio waves to detect objects in the vehicle's surroundings. They are commonly used in the automotive industry for object avoidance.

Working principle Similar to LiDAR sensors, radar sensors emit radio waves and measure the time it takes for the waves to reflect back to the sensor. By calculating the time difference, the sensor can determine the distance to the object based on the speed of light. Radar sensors can also estimate the object's velocity based on the Doppler effect [6].

Advantages and Limitations Radar sensors are less sensitive to adverse weather conditions than LiDAR sensors, making them more reliable in rain, snow, or fog. [1, 7]. However, radar sensors have a lower resolution than LiDAR sensors, which can limit their ability to detect small objects or provide detailed information about the object's shape.

Image-Based Perception

Image based perception systems use cameras to capture images of the vehicle's surroundings and are the principal sensor in some companies like Tesla [8]. These sensors can be used as single cameras for object detection and classification or in pairs to provide stereo vision, which allows for depth estimation and 3D reconstruction of the scene. These type of sensors can also be used for optical flow computation and the ego motion estimation through visual odometry

Monocular vision Monocular vision systems use a single camera to capture images of the vehicle's surroundings. These systems are commonly used for object detection and classification tasks, such as detecting pedestrians, cyclists, or other vehicles [1]. Monocular vision systems are less expensive than stereo vision systems and can be easily integrated into existing vehicles. However, they have limitations in depth perception and 3D reconstruction, which can make it challenging to estimate the distance to objects accurately.

Stereo vision Stereo vision systems use two cameras to capture images of the vehicle's surroundings and use the pinhole camera model [9] to estimate the depth of objects in the scene. By comparing the images from the two cameras, the system can calculate the disparity between corresponding points in the images and use this information to estimate the distance to the objects.

Advantages and Limitations Image-based perception systems have the advantage of providing a rich and detailed information about the environment. Cameras can capture color information, texture, and shape of objects, which can be useful for object detection and classification tasks. In addition, cameras are less expensive than LiDAR sensors and can be easily integrated into existing vehicles. However, image-based perception systems are sensitive to adverse weather conditions, such as rain, snow, or fog, which can affect the quality of the images and limit the system's performance. The algorithms used for object detection and tracking in image-based perception systems can also be computationally intensive, which can affect the system's real-time performance.

In the following table a summary of the advantages and limitations of the different sensor types used in autonomous vehicles is provided:

Table 2.1: Summary of sensor types used in autonomous vehicles

Sensor type	Advantages	Limitations
LiDAR	Accurate 3D representation	Expensive and sensitive to adverse weather conditions
Radar	Works in bad weather, detects velocity	Low spatial resolution
Cameras	Rich scene details, color information, Cheaper than the other considered sensors	Depth perception is limited, affected by lighting, expensive computation cost

While sensors provide raw data about the environment, perception algorithms are responsible for interpreting this information to detect and track moving objects. These algorithms can be broadly classified into learning-based and deterministic approaches, as discussed in the following sections.

2.2 Algorithms for moving object detection

The perception algorithms can also be classified into two main categories: learning-based and deterministic approaches. Learning-based approaches use machine learning techniques, such as deep learning, and are data driven. In the other side, deterministic approaches use mathematical models to estimate the state of the objects in the scene. In the following sections we will provide an overview of the most common algorithms used for moving object detection in autonomous vehicles.

Learning-Based Approaches

Learning-based approaches have gained popularity in recent years. These methods use convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers to process the data coming from cameras, LiDARs, and radar systems. In the work done by Wen and Jo [10], they provide a comprehensive review of the different learning-based approaches used in autonomous vehicles, which can be summarized in the following figure:

Some limitations of learning-based approaches are the need for extensive training data, which can be expensive and time-consuming to collect. These methods can also struggle to generalize in novel and untrained scenarios, which can be problematic in safety-critical situations where the vehicle must make quick decisions based on the perceived environment.

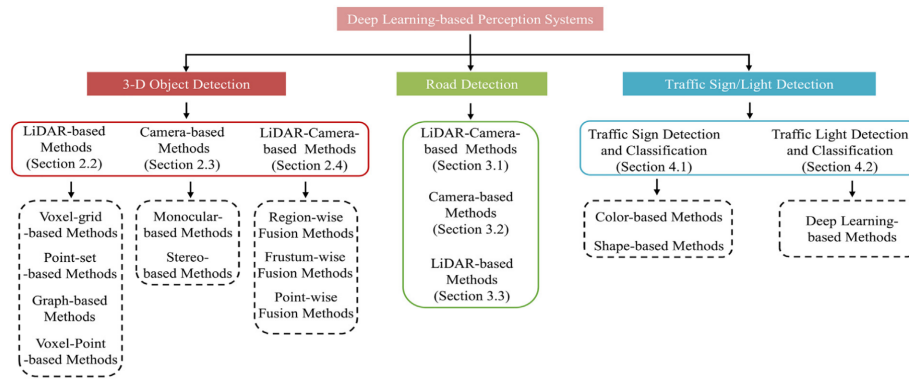


Figure 2.1: Learning-based perception methods. Source: [10]

Deterministic Approaches

Deterministic approaches use mathematical models to estimate the state of the objects in the scene.

Optical flow Optical flow is a technique used to estimate the motion of objects in the scene by tracking the movement of pixels between consecutive frames. This method is based on the assumption that the intensity of a pixel remains constant over time, which allows the system to track the movement of objects based on the changes in pixel intensity [11]. Some of the most common optical flow algorithms are the Lucas-Kanade method [12] and the Horn-Schunck method [**flow_schunck**].

6D Vision 6D vision is a deterministic perception framework that combines stereo vision and motion tracking to estimate the 3D position and velocity of objects in the scene. Originally introduced by Franke et al [13] this method provides a direct motion estimation without requiring object segmentation. The key principles of 6D Vision are:

1. **Stereo disparity estimation:** Calculate the disparity between corresponding points in the stereo images to estimate the depth of objects in the scene.
2. **Optical flow tracking:** Track the movement of feature points in the scene between consecutive frames.
3. **Kalman filtering:** Use Kalman filters to estimate the 6D state vector (position and velocity) of the feature points in the scene.

2.3 State of the Art in the FTM

The Institute of Automotive Technology (FTM) at the Technical University of Munich has conducted previous research in the field of environment perception for autonomous and automated vehicles. This section provides an overview of previous work carried out at the FTM, particularly regarding the EDGAR research vehicle and the development of deterministic perception systems based on 6D vision.

2.3.1 EDGAR Research Vehicle

The EDGAR (Excellent Driving Garching) research vehicle is a state-of-the-art autonomous driving platform developed at FTM [14]. It serves as a testbench for autonomous driving research, including perception, planning, control, and sensor fusion. The vehicle is equipped with a multi-sensor setup, comprising:

1. **Stereo and monocular cameras:** For image-based perception and object detection
2. **LiDAR sensors:** For 3D mapping and obstacle detection
3. **Radar sensors:** For object avoidance and velocity estimation
4. **GPS/IMU:** For localization and motion tracking

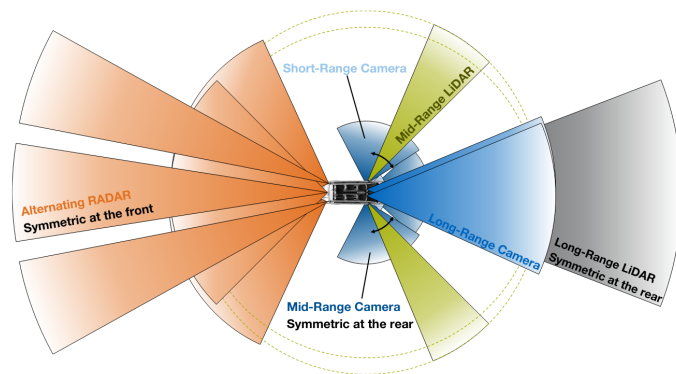
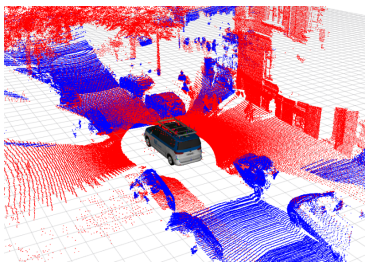


Figure 2.2: EDGAR Vehicle Sensors

2.3.2 Previous Work in the FTM

This project builds upon previous work conducted at the FTM in the field of environment perception for automated vehicles with 6D vision. Some of the key contributions include:

1. **Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren** [15]: Bauer in his master's thesis developed a deterministic system for object detection based on the 6D vision of Daimler [13]. The system was implemented in C++ using the ADTF framework Automotive Data and Time-Triggered Framework and was capable of detecting and tracking objects in the vehicle's surroundings with some real time limitations.
2. **Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen**
Some time later in [16] Rotar continued Bauers work by implementing the system in GPU using CUDA to speed up the computationally intensive tasks.
3. **Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles:**
Finally Pastor [17] started porting the system to ROS1 to facilitate the integration with other modules in the EDGAR vehicle. The system was implemented without CUDA GPU acceleration and was not deployed in the EDGAR vehicle. The system also has some problem with the object detection.

Building upon these previous contributions, this project aims to further improve deterministic perception by addressing key limitations found in prior implementations. In particular, this work will focus on enhancing

computational efficiency, real-time feasibility, and system deployment within ROS2. These improvements will be detailed in the next section on objectives.

2.4 Stereo camera technology

Short review
on stereo cam-
era technol-
ogy.

2.5 Methods for software containerization

Short review
on methods for
software con-
tainerization.

2.6 Requirements for the Thesis

Short review
on the require-
ments for the
thesis.

3 Methods and Implementation

This chapter outlines the methodological and technical approaches that form the foundations of this project. It begins with a high level architecture of the system, providing insight into how the different modules interact to enable real-time perception and criticality assessment for automated vehicles.

Following this, each component of the system is examined in detail, from the underlying algorithms to the some of the implementation details. Emphasis is placed on the design choices made to ensure robustness, real-time performance, and modularity.

By decomposing the system into well-defined modules, this architecture enables both scalability and flexibility for experimentation, while maintaining consistent performance in real-world conditions. It also facilitates further development and seamless integration with other components of the EDGAR vehicle's software stack.

3.1 System Architecture

The system is conceptually divided into two main pipelines: the perception pipeline and the criticality pipeline. The **perception pipeline** processes raw sensor data captured by the stereo camera to build a dynamic representation of the environment and estimating the position and speed vectors of the moving objects. The output of this pipeline is a list of object in the scene that is later used by the **criticality pipeline** to compute the criticality score of the detected objects.

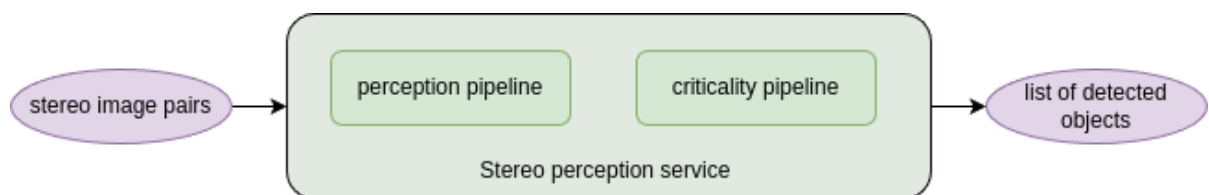


Figure 3.1: High level overall system

3.1.1 Perception pipeline

The perception pipeline will be responsible for detecting moving objects and estimating their position and current velocity. The pipeline consists of several submodules as illustrated in figure 3.2.

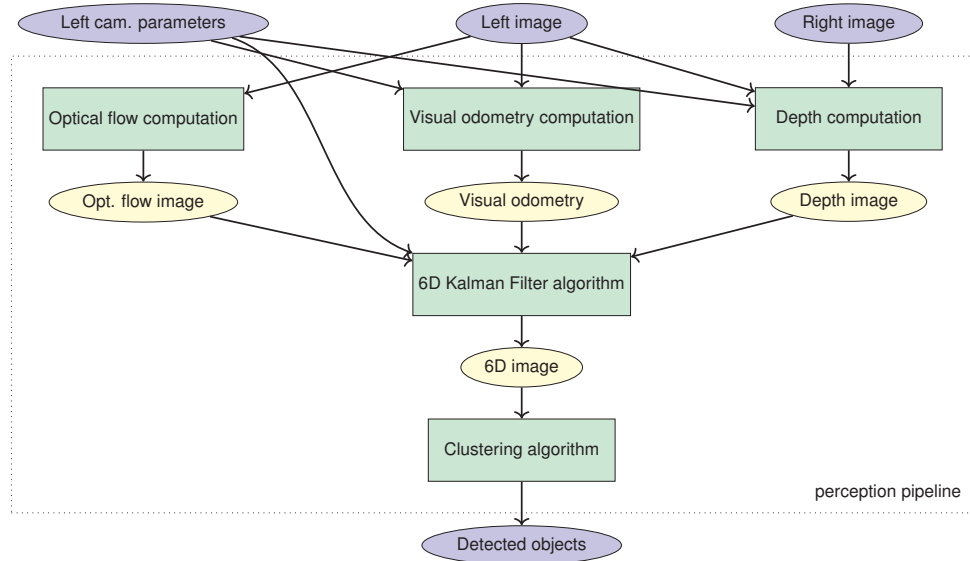


Figure 3.2: High level perception pipeline

As shown in the figure, the perception pipeline comprises the following core modules:

1. **Optical flow computation:** Calculates a dense optical flow field from the left image sequence using the Farneback algorithm.
2. **Visual odometry:** Estimates the ego-motion of the vehicle using stereo image sequences
3. **Depth computation:** Computes a dense depth image from the stereo image pair using a semi-global matching (SGM) algorithm.
4. **Kalman filter:** Central module of the perception pipeline. It uses depth and optical flow data to estimate the 6D state (position and velocity) of a sparse set of points over time.
5. **Object detection:** Applies a clustering algorithm (DBSCAN) to group similar 6D points into detected objects.

3.1.2 Criticality pipeline

The criticality pipeline (Figure 3.3) assesses the risk posed by each detected object. Based on the output of the perception pipeline and the current trajectory of the ego-vehicle, it computes criticality metrics such as the Time-to-Collision (TTC) [18]. These metrics serve as indicators of potential hazards in the environment.

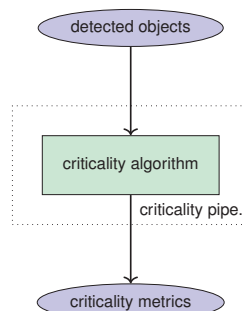


Figure 3.3: High level criticality pipeline

3.1.3 Repository structure

The project is implemented as a monorepo, consolidating all system components into a single, unified repository to simplify development, integration, and deployment.

The repository is structured into three main directories:

- **src:** Contains all ROS 2 source code. It is further divided into:
 - **launch_packages:** Includes ROS 2 launch files used to start multiple components together.
 - **perception_pipeline:** Contains the core modules of the perception pipeline, including **camera_info_publisher**, **stereo_computation**, **optical_flow_computation**, **kalman_filter**, and **object_detector**.
 - **criticality_pipeline:** Contains the **ttc_calculator** package responsible for computing Time-to-Collision and other criticality metrics.
 - **utils:** Provides auxiliary packages and shared message definitions in **stereo_perception_msgs**.
- **scripts:** Includes helper bash scripts and Dockerfiles used to build, run, and deploy the system.
- **config/dds:** Contains configuration files related to middleware settings such as DDS profiles.

An overview of this structure is shown in Figure 3.4.

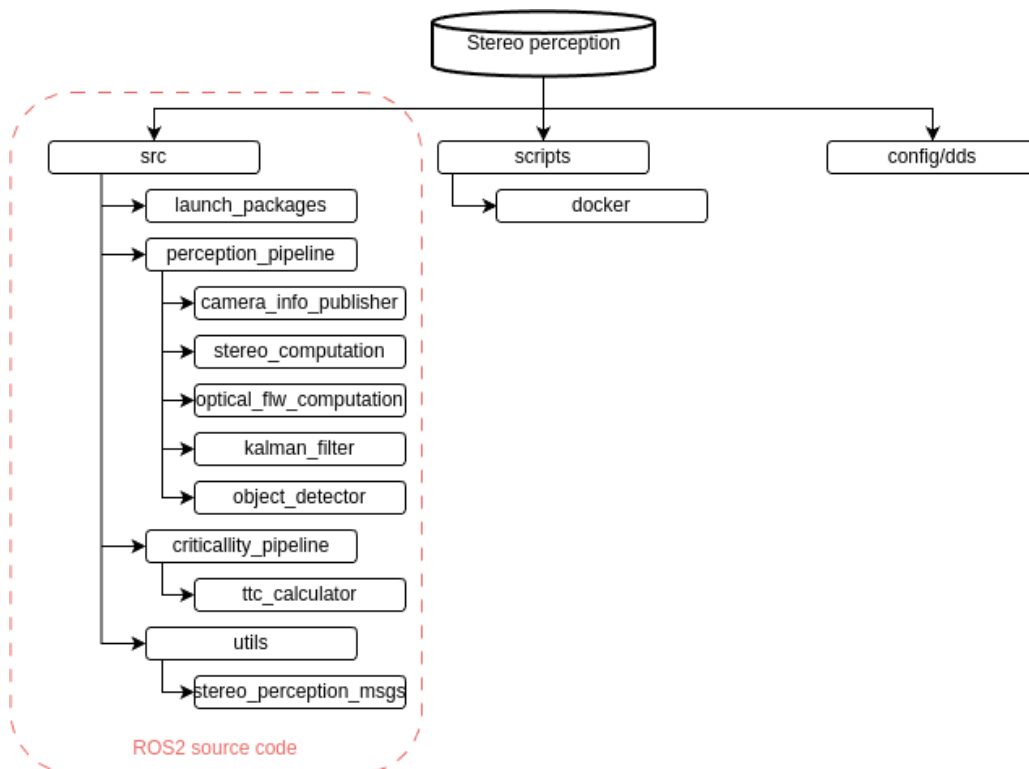


Figure 3.4: Overview of the repository structure

3.1.4 Containerization

Docker is a platform for developing, shipping, and running applications in lightweight, isolated environments called containers. Containers package the application code together with its dependencies and runtime, ensuring consistent behavior across different systems and deployment contexts.

To ensure reproducibility, portability, and streamlined deployment, the entire system is encapsulated within a single Docker container. This design choice is driven by both functional and performance considerations.

Firstly, the system is architected as a unified perception service that processes synchronized stereo camera inputs to produce a list of detected objects along with their associated criticality scores. Given the tight coupling and interdependencies among the modules, segregating components into multiple containers would introduce unnecessary inter-process communication overhead.

Secondly, employing a single container mitigates latency introduced by serialization and network layers between containers. This is particularly crucial in real-time systems, where maintaining low processing latency is imperative. Running all modules in the same memory space allows for efficient data exchange and eliminates potential bottlenecks associated with inter-container messaging.

This containerization strategy also simplifies integration with the research platform, facilitating easier deployment, logging, and testing within a controlled and reproducible environment.

Explain how
it was built:
OpenCV,
CUDA, ROS2,
etc.

3.2 Perception pipeline: Stereo image computation

The stereo image computation module is responsible for generating a dense depth map from a pair of rectified stereo images and its calibration parameters. This component is particularly relevant when working with datasets or sensors that provide raw stereo image pairs but no precomputed depth information. Although this module is not used in the final deployment of the system on the EDGAR research vehicle, since its stereo camera setup provides depth images directly—it plays a crucial role in training, testing, and benchmarking the system using external datasets. It can also be used in the future to integrate other stereo camera sets that do not provide depth images.

3.2.1 Mathematical Background: Pinhole Camera Model and Disparity

In stereo vision, depth estimation relies on the triangulation of corresponding points in a pair of images captured from slightly different viewpoints. The pinhole camera model (figure: 3.5) is commonly used to describe the relationship between 3D world coordinates and 2D image coordinates. In this model, a 3D point $\mathbf{P} = (X, Y, Z)$ is projected onto the image plane as a 2D point $\mathbf{p} = (x, y)$ using the camera's intrinsic parameters ([9]). These parameters include the focal length f and the principal point (c_x, c_y) , which define the camera's optical center and field of view.

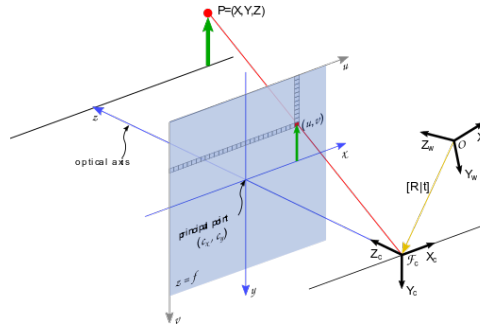


Figure 3.5: Pinhole camera model. Source: Nvidia

The relationship between the 3D point and its 2D projection is given by:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (3.1)$$

where f_x and f_y are the focal lengths in pixels, and (c_x, c_y) is the principal point.

In a typical stereo setup, the two cameras are displaced horizontally along the x-axis, with their optical axes kept parallel. This physical separation between the cameras is known as the **baseline** B , as seen in the figure 3.6 left. The baseline is a key parameter in stereo vision, as it directly influences the triangulation accuracy: a larger baseline improves depth resolution, especially for distant objects, but also increases the difficulty of finding correct correspondences due to greater viewpoint differences.

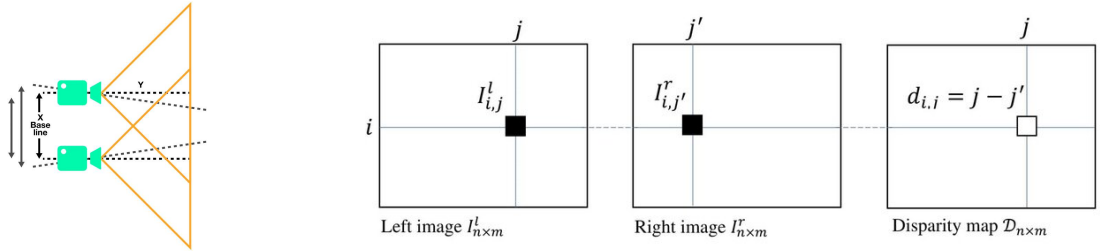


Figure 3.6: Stereo camera setup (left) and disparity concept (right). Source: Framos, [19].

Thanks to this horizontal arrangement, the correspondence search simplifies to a one-dimensional problem along the image rows. After rectification, corresponding points between the two images lie along the same horizontal line (epipolar line), and the **disparity** d (figure 3.6 right) between corresponding points is defined as:

$$d = x_{\text{left}} - x_{\text{right}} \quad (3.2)$$

Disparity is inversely proportional to the depth of the observed point. The relation between disparity and depth Z is given by:

$$Z = \frac{f \cdot B}{d} \quad (3.3)$$

3.2.2 Stereo Matching Algorithms

Stereo matching algorithms are responsible for computing the disparity map between a pair of rectified stereo images. Given that disparity directly relates to depth, these algorithms are a critical component of the stereo vision pipeline. Their main task is to find, for each pixel in the left image, the corresponding pixel in the right image along the same horizontal line.

Several approaches exist to solve this correspondence problem, each with different trade-offs between computational complexity, robustness, and accuracy. Some of the most common implemented stereo matching algorithms include:

- **Block Matching (BM):** A local method that compares fixed-size blocks between the left and right images. While computationally efficient, it often struggles with textureless regions and depth discontinuities ([20]).
- **Semi-Global Matching (SGM):** An algorithm that balances accuracy and computational efficiency by aggregating matching costs along multiple paths through the image ([21]).

Comparative studies ([22]) have shown that SGM offers a good trade-off between accuracy and performance, making it suitable for real-time applications where both speed and quality are essential.

3.2.3 Semi-Global Matching (SGM)

Semi-Global Matching (SGM) is a stereo matching algorithm that offers a compromise between the accuracy of global optimization methods and the computational efficiency of local methods. It was first introduced by Hirschmüller [21] and has become widely implemented in real-time applications.

The underlying principle in SGM is to minimize a global energy function that imposes continuity in the disparity map, having strong depth discontinuities along object boundaries. The energy function $E(D)$ for a disparity map D is defined as:

$$E(D) = \sum_{\mathbf{p}} \left(C(\mathbf{p}, D_{\mathbf{p}}) + \sum_{\mathbf{q} \in \mathcal{N}_{\mathbf{p}}} P_1 \cdot \delta(|D_{\mathbf{p}} - D_{\mathbf{q}}| = 1) + P_2 \cdot \delta(|D_{\mathbf{p}} - D_{\mathbf{q}}| > 1) \right) \quad (3.4)$$

where:

- $C(\mathbf{p}, D_{\mathbf{p}})$ is the matching cost at pixel $\mathbf{p}$ for disparity $D_{\mathbf{p}}$.
- $\mathcal{N}_{\mathbf{p}}$ denotes the set of neighboring pixels.
- P_1 and P_2 are penalty terms for disparity changes of 1 and greater than 1, respectively.
- δ is the Kronecker delta function.

The algorithm operates in two main steps:

1. Cost computation: For each possible disparity, a matching cost calculation happens between the left picture plus the right picture at each pixel. This calculation uses methods like the Sum of Absolute Differences or measurements that use census transform.

2. Cost aggregation: SGM does not do a full 2D global optimization. That process demands a lot of computation, so it is not practical for systems that need to run in real time. SGM comes close to global reasoning when it combines costs along several 1D paths. Costs are put together along either 4 or 8 directions from each pixel. A dynamic programming method finds the lowest local cost in each direction. It also adds penalties for significant changes in disparity.

Once the cost values are added together from all directions, the disparity picked for each pixel is the one that has the lowest combined cost.

This balance between accuracy and computational efficiency makes SGM highly attractive for embedded and real-time applications.

3.2.4 ROS2 Implementation

The SGM algorithm has been integrated into the pipeline with a dedicated ROS2 package called `stereo_computation`. It contains a node that subscribes to three topics: the rectified left and right camera images, and the camera info topic of the left camera. After receiving synchronized input, the node computes both the disparity map and the depth map, which are then published on two separate ROS2 topics for downstream processing.

The stereo matching algorithm was implemented using the OpenCV library. In particular, the `cv::cuda::StereoSGM` class was chosen to compute the disparity map. This class offers a CUDA-accelerated version of the Semi-Global Matching algorithm, enabling the computations to be performed directly on the GPU. Taking advantage of GPU acceleration is crucial to meet the real-time performance requirements of the perception pipeline.

The OpenCV implementation of SGM is highly optimized and designed to exploit the parallel architecture of modern GPUs, significantly reducing the runtime compared to CPU-based methods. After computing the disparity map, the depth map is generated by applying the classical pinhole camera equation:

$$Z = \frac{f \cdot B}{d} \quad (3.5)$$

where Z is the depth value, f is the focal length of the camera, B is the baseline distance between the two cameras, and d is the disparity.

This node architecture ensures modularity within the perception pipeline and allows easy integration with other modules, such as the Kalman filtering and object detection components. Furthermore, by publishing both the disparity and depth images, the system maintains flexibility for future extensions or algorithm improvements that may require direct access to raw disparity data.

To make the tuning and implementation easier, the following ROS2 parameters were added to the node:

ROS2 parameter	Definition
in_left_image_topic	Topic name for the left input grayscale image
in_right_image_topic	Topic name for the right input grayscale image
in_camera_info_topic	Topic name for the camera calibration information
out_depth_image_topic	Output topic for the computed depth image
out_disparity_image_topic	Output topic for the computed disparity image
min_disparity	Minimum disparity to consider for the disparity calculation
num_disparities	Number of disparities (disparity range) to compute
P1	Control parameter for the SGM algorithm (penalty for small disparity changes)
P2	Control parameter for the SGM algorithm (penalty for small disparity changes)
uniqueness_ratio	Threshold for uniqueness of matching points (higher means stricter matching)
speckle_window_size	Size of the window used for speckle filtering
speckle_range	Maximum disparity range for speckle filtering
pre_filter_cap	Pre-filtering parameter for the image to improve disparity computation
use_sim_time	Flag to use simulation time instead of system time

and the node can be added to a launch file as seen in the following example:

Code 3.1: Stereo computation launch

```

1  launch_ros.actions.Node(
2      package='stereo_computation',
3      executable='stereo_computation_node',
4      output='screen',
5      parameters=[
6          {"in_left_image_topic": input_gray_left_topic},
7          {"in_right_image_topic": input_gray_right_topic},
8          {"in_camera_info_topic": input_camera_info_topic
9      },
10     {"out_depth_image_topic": "perception_pipeline/
depth_image"},
11     {"out_disparity_image_topic": "
perception_pipeline/disparity"},
12     {"min_disparity": 0},
13     {"num_disparities": 256},
14     {"P1": 30},
15     {"P2": 200},
16     {"uniqueness_ratio": 15},
17     {"speckle_window_size": 50},

```



```
17         {"speckle_range": 2},
18         {"pre_filter_cap": 31},
19         {"use_sim_time": use_sim_time}
20     ],
21     ),
```

3.3 Perception pipeline: Optical flow computation

This section describes the optical flow computation module within the perception pipeline. This module is responsible for estimating the relative motion of the pixels of the image between two consecutive frames.

3.3.1 Optical flow concept

The concept of Optical Flow, first introduced by James Gibson in the 1950s ([23]) refers to the displacements of intensity patterns. In our case it represents the relative motion of the pixels between two consecutive frames of a video stream. The output is a 2D vector field where each vector describes the position of a pixel or a small region in the current frame relative to its position in the previous frame.

This module is crucial for estimating the information of how the pixels in the image are moving, which is one of the essential inputs to the Kalman Filter stage.

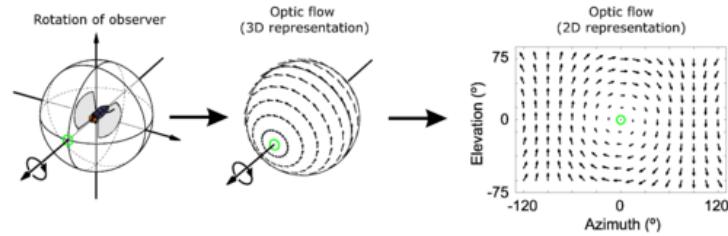


Figure 3.7: Optical flow

Optical flow estimation relies on the assumption that the brightness of a pixel remains constant over time [24, 25], which allows the system to track the movement of objects based on the changes in pixel intensity. Mathematically, this assumption leads to the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \quad (3.6)$$

Where:

- I_x and I_y are the spatial gradients of the image intensity in the x and y directions.
- I_t is the temporal gradient of the image intensity (the difference between two consecutive frames).
- u and v are the optical flow components in the x and y directions.

Since there is one equation and two unknowns, the optical flow constraint equation is underdetermined. To solve this problem, additional constraints or assumptions are needed, leading to different optical flow algorithms.

3.3.2 Optical flow algorithms

Optical flow methods can be classified into two main categories: sparse and dense optical flow.

Sparse optical flow methods focus on estimating motion vectors at a discrete set of key points in the image. These points are typically features such as corners or edges that can be tracked reliably across frames. In contrast, **dense optical flow** methods aim to estimate a motion vector for every pixel in the image, providing a complete 2D displacement field.

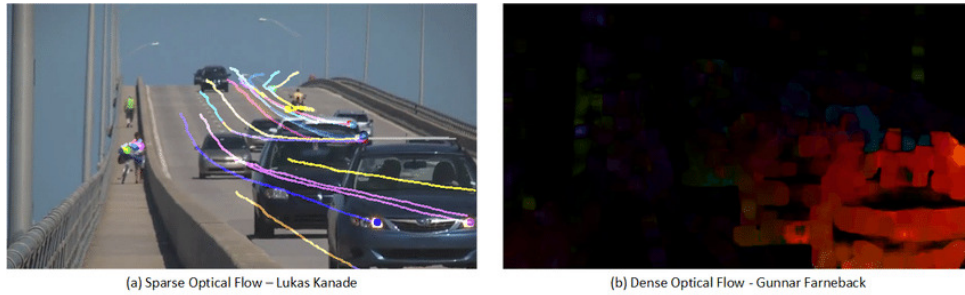


Figure 3.8: Sparse vs dense optical flow. Source: [26]

Recently, deep learning-based methods have gained popularity and demonstrated impressive performance for optical flow estimation. Models such as FlowNet [27] and RAFT [`opt_flwRAFT`] leverage convolutional neural networks (CNNs) to learn spatial and temporal features from large datasets, enabling them to predict optical flow fields with high accuracy. However, these methods often require significant computational resources and may suffer from poor generalization to unseen environments.

For these reasons, and due to the necessity of robustness and real-time performance, this work focuses on traditional deterministic optical flow algorithms. These classical methods also offer the advantage of being widely implemented and optimized in existing libraries, which significantly accelerates development and integration time.

Some of the most common optical flow algorithms include [11, 28]:

- **Lucas-Kanade method [12]:**

Introduced in 1981 by Lucas and Kanade, this is a sparse optical flow method based on the assumption of constant optical flow within a small local neighborhood. It solves the optical flow constraint equation using a least-squares approach. This method is simple and computationally efficient, but it is sensitive to large motions and requires good texture in the tracked regions.

- **Horn-Schunck method [29]:**

Proposed by Horn and Schunck, this method computed dense optical flow by introducing a global smoothness constraint. It formulates flow estimation as the minimization of an energy function that combines the brightness constancy constraint and a smoothness term. While robust to moderate noise and capable of estimating full dense flow, the method can oversmooth motion boundaries and struggles with large displacements.

- **TV-L1 method [30]:**

This approach refines the variational method of Horn and Schunck by introducing a total variation (TV) regularization term and an L1 norm for the data term. The resulting algorithm is more robust and was implemented in the previous work of Bauer [15]. Nevertheless, it is computationally expensive and may not be the best choice for real-time applications.

- **Farneback method [31]:** Farneback introduced a dense optical flow method based on local polynomial expansions. The idea is to approximate the neighborhood of each pixel with a quadratic polynomial and then estimate how the polynomials change between frames.

This method offers a good trade-off between accuracy and computational efficiency. When accelerated using GPUs (e.g. via OpenCV CUDA modules), it can achieve real-time performance even on high-resolution images.

In this work, the Farneback method has been selected due to its computational efficiency, robustness for medium-range motions, and the availability of an optimized CUDA implementation.

3.3.3 Farneback algorithm

The Farneback optical flow method is a dense flow estimation algorithm based on polynomial expansion of local image neighborhoods. Its core idea is to approximate the intensity variation in a small image region using a quadratic polynomial, and then infer the motion between frames by analyzing how these polynomials deform over time.

The first step is to approximate the image intensity function $I(x, y)$ in a local neighborhood using a second-order Taylor expansion:

$$I(x, y) \approx x^T A x + b^T x + c \quad (3.7)$$

where:

- $x = (x, y)^T$ represents the spatial coordinates,
- A is a symmetric 2×2 matrix encoding the second-order spatial derivatives,
- b is a 2×1 vector representing the first-order spatial derivatives,
- c is a scalar term corresponding to the local intensity offset.

This local polynomial approximation can be efficiently computed over the entire image using Gaussian-weighted least squares fitting, a process known as polynomial expansion.

When the scene undergoes a pure translation $d = (d_x, d_y)^T$ between two consecutive frames, the transformed intensity function $I'(x)$ can be related to the original by:

$$I'(x) = I(x - d) \quad (3.8)$$

Substituting the polynomial approximation yields:

$$I'(x) \approx (x - d)^T A (x - d) + b^T (x - d) + c \quad (3.9)$$

Expanding the right-hand side leads to:

$$I'(x) \approx x^T A x + (b - 2Ad)^T x + (d^T A d - b^T d + c) \quad (3.10)$$

Comparing terms, we observe that:

$$A' = A \quad (3.11)$$

$$b' = b - 2Ad \quad (3.12)$$

$$c' = d^T A d - b^T d + c \quad (3.13)$$

where A' and b' represent the new polynomial coefficients after displacement.

Thus, the displacement d can be estimated by solving the linear system:

$$2Ad = b - b' \quad (3.14)$$

assuming that A is non-singular and well-conditioned.

In practice, direct estimation is complicated by noise, local deformations, and the limited validity of polynomial approximations. To address these issues, the following refinements are applied:

- Estimate the displacement over a local window using Gaussian weighting to prioritize central pixels.
- Aggregate local estimates using weighted least squares to improve robustness against noise.
- Employ a multi-scale coarse-to-fine pyramid approach to capture both small and large displacements effectively.

Coarse-to-fine pyramid approach

Large displacements between frames can cause local matching failures if only a single resolution is used. To handle this, the Farneback method employs a coarse-to-fine multi-scale pyramid strategy [32].

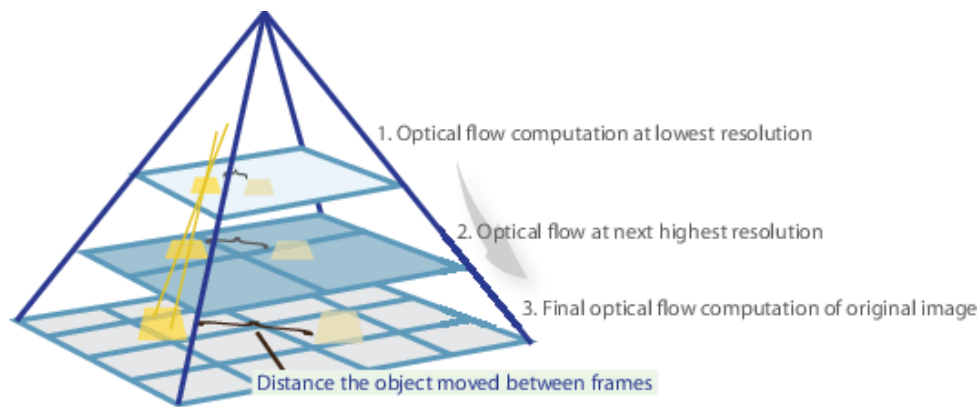


Figure 3.9: Coarse-to-fine pyramid approach. Source: Matlab

In the pyramid approach:

- The original images are repeatedly downsampled by a scaling factor (typically 0.5), producing several lower-resolution versions.
- Optical flow is first estimated at the coarsest (smallest) scale, where large motions appear reduced and easier to estimate.
- The initial flow estimate is then propagated to the next finer scale, where it is refined using the higher-resolution image data.
- This process continues until the finest scale (the original resolution) is reached, producing a dense and detailed optical flow field.

The pyramid-based coarse-to-fine strategy is crucial for ensuring that large motions do not overwhelm the local polynomial approximation, maintaining both the accuracy and robustness of the Farneback method.

This hierarchical refinement significantly enhances the capability of the algorithm to deal with large inter-frame displacements, dynamic scenes, and varying object velocities.

3.3.4 Implementation

This algorithm has been implemented in a ROS2 package called `optical_flow_computation`. This package contains a node responsible for computing the optical flow between two consecutive images. The node subscribes to two topics: the first for the left image and the second for the right image. Once it receives both images, it computes the optical flow and publishes the result on an output topic.

Regarding the algorithm implementation, the `cv::cuda::FarnebackOpticalFlow` class from OpenCV was used. This class provides a CUDA-accelerated version of the Farneback algorithm, allowing computations to be performed directly on the GPU. The OpenCV implementation of the Farneback algorithm is highly optimized and designed to leverage the parallel architecture of modern GPUs, significantly reducing execution time compared to CPU-based methods. This class is included in the header `opencv2/cudaoptflow.hpp`, which generally requires manually building OpenCV with CUDA support.

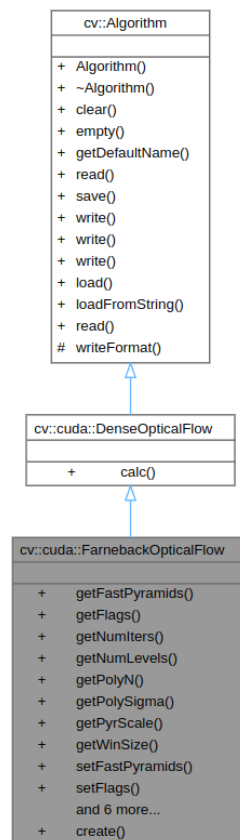


Figure 3.10: CUDA Farneback class. Source: OpenCV

To facilitate tuning and configuration, the following ROS2 parameters were added to the node:

Table 3.1: ROS2 parameters for the optical flow computation node

ROS2 parameter	Definition
image_topic	Input topic name for raw RGB images used in optical flow estimation
optical_flow_topic	Output topic name where the computed optical flow is published
pyr_scale	Scaling factor between pyramid levels
levels	Number of pyramid layers, including the original image
winsize	Averaging window size for smoothing derivatives; larger values improve robustness but may oversmooth motion boundaries
iterations	Number of iterations at each pyramid level; higher values improve convergence in complex motion areas
poly_n	Size of the pixel neighborhood used for polynomial expansion
poly_sigma	Standard deviation of the Gaussian used to smooth derivatives before polynomial expansion
flags	Operation flags passed to the Farneback function
use_sim_time	Boolean indicating whether to use simulation time (/clock) instead of system time

The node can be added to a launch file as shown in the following example:

Code 3.2: Optical flow computation launch

```

1 launch_ros.actions.Node(
2     package='optical_flow_computation',
3     executable='optical_flow_node',
4     output='screen',
5     parameters=[
6         {"image_topic": input_gray_left_topic},
7         {"optical_flow_topic": "perception_pipeline/
optical_flow"},
8         {"pyr_scale": 0.5},
9         {"levels": 3},
10        {"winsize": 15},
11        {"iterations": 3},
12        {"poly_n": 5},
13        {"poly_sigma": 1.2},
14        {"flags": 0},
15        {"use_sim_time": use_sim_time}
16    ],
17 ) ,

```

The output image is published on the topic `/perception_pipeline/optical_flow`. This image is published as a `sensor_msgs/Image` message with a 32FC2 encoding format, representing a 32-bit floating-point optical flow image with two channels. The first channel corresponds to the displacement along the x-axis, and the second channel corresponds to the displacement along the y-axis.

In addition to the flow image, two debug images are published on the `/debug` topic to facilitate debugging and result visualization. One debug image shows the original image with optical flow vectors superimposed, and the other displays an RGB image where the intensity represents the magnitude of the optical flow vector and the color represents its direction.

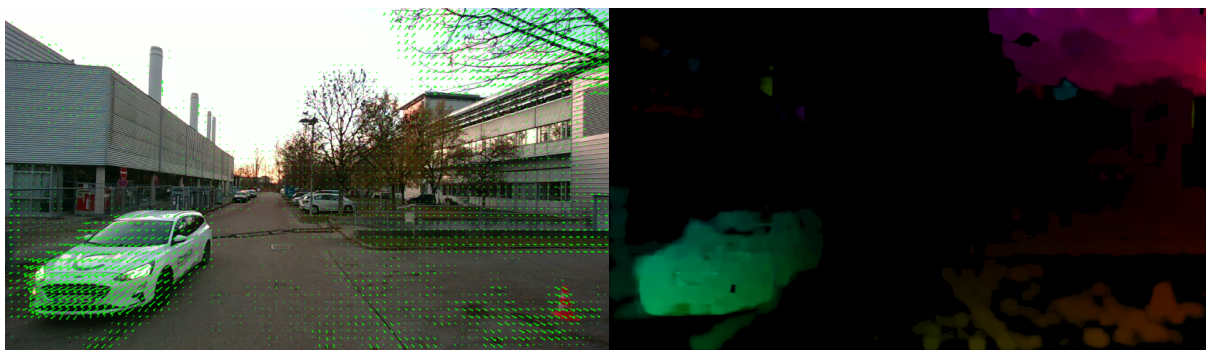


Figure 3.11: Optical flow debug images. Left: Optical flow vectors superimposed on the original image. Right: RGB image where the intensity represents the magnitude and the color represents the direction of the optical flow vector. Source: own work

These additional debug outputs greatly facilitate the qualitative evaluation of the optical flow results and enable faster identification of potential errors during the development and testing phases.

3.4 Perception pipeline: Ego motion estimation

In the context of autonomous and automated vehicles, estimating the ego motion is crucial to understand how the vehicle is moving in the environment. In the scope of this work, ego motion is defined as the relative motion of the vehicle with respect to the environment. Specifically, the information needed is the 3D position of the camera in one frame, relative to the previous frame. This information is essential for the Kalman Filter, since the optical flow output is relative to the camera frame, and the Kalman Filter requires knowledge of the camera's position in the world frame to properly compensate for its motion.

For this purpose, visual odometry (VO) techniques are used to estimate the required information using the stereo setup input. Incorporating the depth information from the stereo camera allows the direct recovery of metric information, providing robustness and accuracy.

This section describes the concept behind visual odometry, the coordinate systems used, the specific algorithmic approach that was implemented, and the implementation details.

3.4.1 Visual odometry concept

Visual odometry (VO) refers to the process of estimating the motion of a camera by analyzing a sequence of images it captures. Originally introduced by Moravec [33] and later formalized by Nistér et al. [34], VO has become a fundamental technique for mobile robotics and autonomous driving [35].

The core idea behind VO is to track visual features across consecutive frames and infer the relative motion of the camera from these observations. The estimated motion provides a continuous estimate of the vehicle trajectory without relying on external localization systems.

Visual odometry methods are generally categorized based on the type of input and processing approach used:

- **Monocular VO:** Utilizes a single camera as input. It is cost-effective but suffers from scale ambiguity, meaning that the absolute scale of the motion (e.g., in meters) cannot be determined without additional information. Monocular VO is often used in conjunction with other sensors (e.g., IMU) to resolve this ambiguity [35].
- **Stereo VO:** Utilizes two synchronized cameras to estimate depth through triangulation, which allows direct recovery of metric scale. This removes the scale ambiguity present in monocular systems, enabling more accurate and robust motion estimation. Stereo VO is especially beneficial in environments where monocular methods struggle, such as in low-texture areas or during large baseline movements, thanks to the additional geometric constraints provided by the known baseline distance between the cameras [35].

Processing approaches are typically divided into two categories:

- **Feature-based methods:** Extract and match features (e.g., corners, edges) across frames, estimating motion from the displacement of these features.
- **Direct methods:** Directly use pixel intensities to estimate motion without explicit feature extraction.

Recently, **deep learning-based visual odometry** approaches have emerged, leveraging convolutional and recurrent neural networks to estimate motion directly from image sequences. These methods aim to improve robustness under challenging conditions, such as illumination changes and textureless environments [35].

This work adopts a **stereo feature-based** approach, leveraging the depth information from the stereo setup and using feature matching to estimate motion robustly. This choice provides a good balance between computational efficiency and robustness against noise, dynamic environments, and varying lighting conditions.

3.4.2 Reference frames and coordinate systems

In this work, several coordinate frames are used to represent the motion of the camera:

- **Previous camera frame:** The coordinate frame attached to the camera at the previous time step.
- **Current camera frame:** The coordinate frame attached to the camera at the current time step.
- **Initial camera frame:** The fixed world reference frame, corresponding to the camera pose at the start of the sequence.

The visual odometry algorithm estimates the relative motion between the previous and current camera frames. This relative transform is then accumulated over time to estimate the pose of the camera with respect to the initial frame.

The coordinate system follows the standard ROS convention for cameras:

- The **z-axis** points forward, along the optical axis.
- The **x-axis** points to the right of the camera.
- The **y-axis** points downwards.

It is important to note that this convention differs from the standard world coordinate system, where typically the x-axis points forward, the y-axis points to the left, and the z-axis points upwards. In our setup, the z-axis aligns with the camera's optical axis (forward), while the x- and y-axes are oriented relative to the image plane.

3.4.3 Implemented algorithm

Overview of the algorithm

The visual odometry algorithm implemented in this work follows a feature-based stereo approach. It estimates the relative motion of the camera by detecting and matching visual features between consecutive frames, reconstructing their 3D positions using the depth image, and solving the camera pose using a Perspective-n-Point (PnP) algorithm with RANSAC for outlier rejection.

A schematic overview of the algorithm steps is presented in Algorithm 1.

Algorithm 1 Stereo Visual Odometry Pipeline**Require:** Grayscale images I_{k-1} , I_k , and depth image D_{k-1} **Ensure:** Estimated relative pose (R, t) between frames

- 1: Detect ORB features in I_{k-1} and I_k using GPU acceleration
- 2: Match ORB features between I_{k-1} and I_k
- 3: Reproject matched keypoints from I_{k-1} into 3D using D_{k-1}
- 4: Solve PnP with RANSAC using 3D-2D correspondences
- 5: Refine the pose estimation with standard solvePnP using inliers
- 6: Estimate covariance based on reprojection residuals
- 7: Apply statistical filtering and optional exponential smoothing
- 8: Output (R, t) , covariance matrix, and debug images

Feature detection and matching

Feature detection and matching constitute the first step in the visual odometry pipeline. In this work, ORB (Oriented FAST and Rotated BRIEF) [36] features are employed due to their balance between computational efficiency and robustness.

ORB is a binary feature descriptor that builds upon two classical methods: FAST (Features from Accelerated Segment Test) [37] for keypoint detection and BRIEF (Binary Robust Independent Elementary Features) [38] for descriptor computation.

FAST is a corner detection algorithm that rapidly identifies points in the image with significant intensity change compared to their surroundings. BRIEF, on the other hand, generates a binary string descriptor by comparing the intensity of random pairs of pixels in a small patch around the keypoint.

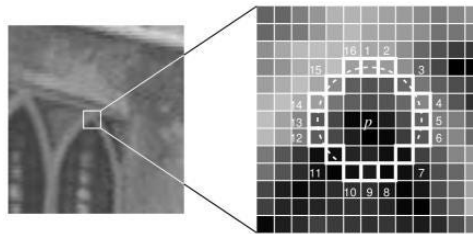


Figure 3.12: FAST algorithm. Source: [39]

ORB enhances these methods by introducing rotation and scale invariance. The orientation of each keypoint is computed to make BRIEF descriptors rotation-invariant, and a multi-scale pyramid is used to handle changes in scale. As a result, ORB features are suitable for real-time applications while maintaining robustness to changes in viewpoint, illumination, and scale.

After detecting keypoints, a descriptor is computed for each one. A descriptor is a compact representation of the local image neighborhood around a keypoint, designed to allow efficient and robust matching of features across frames.

The feature extraction and matching process is performed on the GPU using OpenCV's CUDA modules. This allows real-time processing even at high image resolutions.

The steps involved are:

- Detect ORB features separately in the previous and current grayscale images.
- Compute binary descriptors for each detected feature.

- Match descriptors between the previous and current frames using brute-force Hamming distance.
- Extract the coordinates of matched keypoints for subsequent motion estimation.

GPU acceleration significantly speeds up both feature detection and descriptor matching, enabling the pipeline to meet real-time requirements.

3D-2D correspondences

Once the feature matching step is completed, the next task is to establish 3D-2D correspondences between the previous and current frames. This is necessary for estimating the camera motion through Perspective-n-Point (PnP) techniques.

For each matched keypoint in the previous frame, the corresponding depth value is retrieved from the depth image associated with that frame. If the depth value is within a valid range, the 2D keypoint is reprojected into a 3D point using the intrinsic parameters of the stereo camera.

The reprojection follows the standard pinhole camera model:

$$X = (u - c_x) \cdot \frac{d}{f_x}, \quad Y = (v - c_y) \cdot \frac{d}{f_y}, \quad Z = d \quad (3.15)$$

where (u, v) are the pixel coordinates of the keypoint, (c_x, c_y) are the coordinates of the principal point, (f_x, f_y) are the focal lengths in pixels, and d is the depth value.

This process results in a set of valid 3D points extracted from the previous frame, and their corresponding 2D projections detected in the current frame. These 3D-2D pairs are later used to estimate the relative pose of the camera.

If the depth value is invalid (e.g., missing or outside the valid range), the corresponding match is discarded to avoid introducing noise into the motion estimation process.

Motion estimation using PnP and RANSAC

Once 3D-2D correspondences are available, the relative camera motion between two frames can be estimated by solving the Perspective-n-Point (PnP) problem [40]. The goal is to compute the rotation $\mathbf{R}$ and translation $\mathbf{t}$ that map the 3D points observed in the previous frame to their 2D projections in the current image.

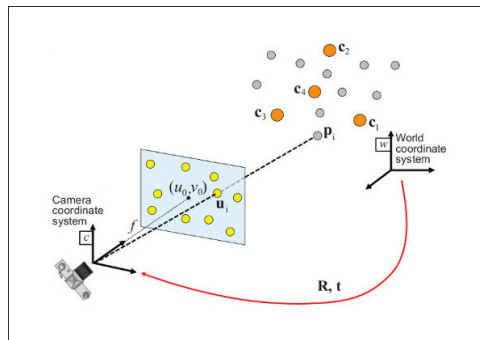


Figure 3.13: Perspective-n-Point (PnP) problem. Source: [41]

Given a set of n correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}$, where $\mathbf{X}_i$ are 3D points in the previous frame and $\mathbf{x}_i$ their 2D image projections in the current frame, PnP finds a transformation that minimizes the reprojection error under the pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K}[\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3.16)$$

Here, $\mathbf{K}$ is the camera intrinsic matrix, and $\mathbf{R}, \mathbf{t}$ represent the relative pose of the current frame with respect to the previous one. The symbol $\sim$ denotes equality up to scale in homogeneous coordinates.

In practice, 3D-2D correspondences may include mismatches or noisy observations. To robustly estimate the pose in the presence of such outliers, RANSAC (Random Sample Consensus) is used in conjunction with PnP.

RANSAC operates by randomly selecting minimal subsets of correspondences, solving the PnP problem for each subset, and evaluating how many correspondences agree with the estimated pose. The solution with the highest number of inliers is selected, and the pose is then refined using all inliers to minimize the overall reprojection error.

In this work, the function `cv::solvePnP` from OpenCV is used to perform motion estimation. This function takes as input the 3D points from the previous frame, their 2D projections in the current frame, and the camera intrinsics, and returns the estimated rotation and translation vectors.

The steps involved are:

1. Project 3D points from the previous frame into the current image using candidate pose.
2. Measure the reprojection error against actual 2D keypoints.
3. Select inliers whose error is below a given threshold.
4. Repeat for several random subsets and select the best solution.
5. Refine the pose using all inliers.

The resulting output consists of:

- `rvec` a rotation vector in axis-angle form.
- `tvec` a 3D translation vector.

The `rvec` returned by `solvePnP` is a Rodrigues rotation vector or axis-angle rotation vector, a compact 3×1 representation of rotation. The direction of the vector indicates the axis of rotation, while its magnitude represents the rotation angle in radians around that axis. This vector can be converted to a rotation matrix using the Rodrigues formula.

This matrix describes the transformation from the previous camera frame to the current one, and can be used for pose chaining, trajectory estimation, or visual odometry back-end integration.

Covariance estimation

After estimating the camera pose using PnP and RANSAC, it is often useful to quantify the uncertainty associated with this estimate. This is particularly relevant when integrating the pose into filtering frameworks, such as an Extended Kalman Filter (EKF), where a covariance is required.

In this work, the pose covariance is approximated based on the reprojection residuals of the inlier correspondences identified by RANSAC. Once the best pose has been selected, all 3D points are projected onto the image using the estimated transform, and the residual reprojection errors are computed as:

$$\mathbf{e}_i = \mathbf{x}_i^{\text{observed}} - \mathbf{x}_i^{\text{projected}}(R, t) \quad (3.17)$$

The residuals are stacked and used to compute a sample covariance matrix:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{e}_i - \bar{\mathbf{e}})(\mathbf{e}_i - \bar{\mathbf{e}})^T \quad (3.18)$$

Although this approach does not yield an exact pose covariance, it provides a reasonable and fast approximation that reflects the quality of the inlier set and the geometric configuration of the matched points.

Motion filtering and smoothing

The pose estimates obtained via PnP are subject to noise due to imperfect keypoint matching, depth errors, or small instabilities in the feature geometry. To improve the consistency of the estimated trajectory, a filtering and smoothing step is applied. Two approaches are combined:

- **Statistical filtering** To detect and remove outliers in both translation and rotation components based on deviation from a local history.
- **Exponential smoothing** To smooth the motion estimate over time using a weighted moving average.

Every component of the translation and rotation vector is separately. A sliding window is stored for each component to compute its mean and standard deviation. New values that exceed a fixed threshold or deviate significantly from the mean are replaced with the mean.

Once filtered, the values are smoothed using exponential smoothing:

$$s_t = \alpha x_t + (1 - \alpha)s_{t-1} \quad (3.19)$$

where s_t is the smoothed value at time t , x_t is the current value, and α is a smoothing factor between 0 and 1.

Rotation vectors are smoothed component-wise in Rodrigues form. Although this is a simplification compared to quaternion-based smoothing on $SO(3)$, it performs well in practice for small motions.

After filtering and smoothing, the pose is converted to a 4x4 transformation matrix and stored as the final motion estimate for the current frame.

3.4.4 ROS2 Implementation

3.4.5 Parameter tuning and outputs

3.4.6 Reference frames and coordinate systems

3.4.7 Visual odometry algorithm

3.4.8 Implementation

3.4.9 Parameter tuning and outputs

3.4.10 Reference frame selection

3.4.11 Implementation

3.5 Perception pipeline: Kalman filter

3.5.1 Kalman filter algorithm

3.5.2 Mathematical model

3.5.3 Implementation

3.6 Perception pipeline: Object detection

3.6.1 DBSCAN clustering

3.6.2 Implementation

3.7 Criticallity pipeline: TTC computation

4 Experiments and Results

4.1 Camera hardware selection

4.2 Experimental setup

4.3 Perception pipeline evaluation

4.3.1 Stereo image computation

4.3.2 Optical flow computation

4.3.3 Ego motion estimation

4.3.4 Kalman filter

4.3.5 Object detection

4.4 Criticallity pipeline evaluation

5 Summary and Conclusion

- High level explanation on how the work will be structured.
- What the perception and criticality pipeline are - Go through each component.

List of Figures

Figure 2.1:	Learning-based perception methods. Source: [10].....	6
Figure 2.2:	EDGAR Vehicle Sensors	7
Figure 3.1:	High level overall system.....	9
Figure 3.2:	High level perception pipeline	10
Figure 3.3:	High level criticality pipeline.....	10
Figure 3.4:	Overview of the repository structure.....	11
Figure 3.5:	Pinhole camera model. Source: Nvidia	13
Figure 3.6:	Stereo camera setup (left) and disparity concept (right). Source: Framos, [19].	14
Figure 3.7:	Optical flow	18
Figure 3.8:	Sparse vs dense optical flow. Source: [26]	19
Figure 3.9:	Coarse-to-fine pyramid approach. Source: Matlab	21
Figure 3.10:	CUDA Farneback class. Source: OpenCV	22
Figure 3.11:	Optical flow debug images. Left: Optical flow vectors superimposed on the original image. Right: RGB image where the intensity represents the magnitude and the color represents the direction of the optical flow vector. Source: own work	24
Figure 3.12:	FAST algorithm. Source: [39]	27
Figure 3.13:	Perspective-n-Point (PnP) problem. Source: [41].....	28
Figure B.1:	Bild im Anhang	xv

List of Tables

Table 2.1:	Summary of sensor types used in autonomous vehicles	5
Table 3.1:	ROS2 parameters for the optical flow computation node	23
Table B.1:	Tabelle im Anhang	xv

Bibliography

- [1] J. Van Brummelen, M. O'Brien, D. Gruyer and H. Najjaran, "Autonomous vehicle perception: The technology of today and tomorrow," *Transportation Research Part C: Emerging Technologies*, vol. 89, pp. 384–406, 2018, DOI: 10.1016/j.trc.2018.02.012.
- [2] BMW. "Level 3 highly automated driving available in the new BMW 7 Series from next spring." en. 2023. Available: <https://www.press.bmwgroup.com/global/article/detail/T0438214EN/level-3-highly-automated-driving-available-in-the-new-bmw-7-series-from-next-spring?language=en> [visited on 02/26/2025].
- [3] Mercedes-Benz. "Drive Pilot," 2025. Available: <https://www.mercedes-benz.de/passengercars/technology/drive-pilot.html> [visited on 02/26/2025].
- [4] Waymo. "Meet the 6th-generation Waymo Driver: Optimized for costs, designed to handle more weather, and coming to riders faster than before," 2024. Available: <https://waymo.com/blog/2024/08/meet-the-6th-generation-waymo-driver> [visited on 02/26/2025].
- [5] Y. Li and J. Ibanez-Guzman, "Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems," *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020, DOI: 10.1109/MSP.2020.2973615.
- [6] J. Leonard, J. How, S. Teller, M. Berger, S. Campbell, et al., "A perception-driven autonomous urban vehicle," *Journal of Field Robotics*, vol. 25, no. 10, pp. 727–774, 2008, DOI: 10.1002/rob.20262.
- [7] R. H. Rasshofer and K. Gresser, "Automotive Radar and Lidar Systems for Next Generation Driver Assistance Functions," in *Advances in Radio Science*, 2005, pp. 205–209, DOI: 10.5194/ars-3-205-2005.
- [8] K. N. Maanyu, D. G. Raj, R. V. Krishna and D. S. B. Choubey, "A Study on Tesla Autopilot," vol. 71, no. 171, 2020.
- [9] R. Szeliski, *Computer Vision: Algorithms and Applications*, (Texts in Computer Science), Cham, Springer International Publishing, 2022, ISBN: 978-3-030-34371-2 978-3-030-34372-9. DOI: 10.1007/978-3-030-34372-9.
- [10] L.-H. Wen and K.-H. Jo, "Deep learning-based perception systems for autonomous driving: A comprehensive survey," *Neurocomputing*, vol. 489, pp. 255–270, 2022, DOI: 10.1016/j.neucom.2021.08.155.
- [11] A. Alfarano, L. Maiano, L. Papa and I. Amerini, "Estimating optical flow: A comprehensive review of the state of the art," *Computer Vision and Image Understanding*, vol. 249, p. 104160, 2024, DOI: 10.1016/j.cviu.2024.104160.
- [12] S. Lucey, R. Navarathna, A. B. Ashraf and S. Sridharan, "Fourier Lucas-Kanade Algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 6, pp. 1383–1396, 2013, DOI: 10.1109/TPAMI.2012.220.
- [13] U. Franke, C. Rabe, H. Badino and S. Gehrig, "6D-Vision: Fusion of Stereo and Motion for Robust Environment Perception," in *Pattern Recognition*, 2005, pp. 216–223, ISBN: 978-3-540-31942-9. DOI: 10.1007/11550518_27.

- [14] P. Karle, T. Betz, M. Bosk, F. Fent, N. Gehrke, et al. "EDGAR: An Autonomous Driving Research Platform – From Feature Development to Real-World Application," arXiv:2309.15492 [cs]. Jan. 2024. DOI: 10.48550/arXiv.2309.15492.
- [15] S. Bauer, "Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren," MA thesis, TUM, Munich, 2014.
- [16] A. Rotar, "Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen," MA thesis, TUM, Munich, 2016.
- [17] C. Pastor, "Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles," BSc. Thesis, TUM, 2021.
- [18] L. Westhofen, C. Neurohr, T. Koopmann, M. Butz, B. Schütt, et al., "Criticality Metrics for Automated Driving: A Review and Suitability Analysis of the State of the Art," *Archives of Computational Methods in Engineering*, vol. 30, no. 1, pp. 1–35, 2023, DOI: 10.1007/s11831-022-09788-7.
- [19] S. Heidari, M. Rogers and P. J. Delmas, "(PDF) An Improved Quantum Solution for the Stereo Matching Problem," in *ResearchGate*, DOI: 10.1109/IVCNZ54163.2021.9653310. Available: https://www.researchgate.net/publication/357424395_An_Improved_Quantum_Solution_for_the_Stereo_Matching_Problem [visited on 04/26/2025].
- [20] J. Karathanasis, D. Kalivas and J. Vlontzos, "Disparity estimation using block matching and dynamic programming," in *Proceedings of Third International Conference on Electronics, Circuits, and Systems*, 1996, 728–731 vol.2, DOI: 10.1109/ICECS.1996.584465. Available: <https://ieeexplore.ieee.org/abstract/document/584465> [visited on 04/26/2025].
- [21] H. Hirschmuller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, DOI: 10.1109/TPAMI.2007.1166.
- [22] R. A. Hamzah and H. Ibrahim, "Literature Survey on Stereo Vision Disparity Map Algorithms," *Journal of Sensors*, vol. 2016, no. 1, p. 8742920, 2016, DOI: 10.1155/2016/8742920. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/8742920> [visited on 04/26/2025].
- [23] D. C. Niehorster, "Optic Flow: A History," *i-Perception*, vol. 12, no. 6, p. 20416695211055766, 2021, DOI: 10.1177/20416695211055766. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8652193/>.
- [24] G. Boesch. "Optical Flow - Everything You Need to Know in 2025," en-US. Oct. 2024. Available: <https://viso.ai/deep-learning/optical-flow/> [visited on 11/12/2024].
- [25] D. Fortun, P. Bouthemy and C. Kervrann, "Optical flow modeling and computation: A survey," *Computer Vision and Image Understanding*, vol. 134, pp. 1–21, 2015, DOI: 10.1016/j.cviu.2015.02.008. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1077314215000429> [visited on 11/13/2024].
- [26] O. Zvorișteanu, S. Caraiman and V. Manta, "Speeding Up Semantic Instance Segmentation by Using Motion Information," *Mathematics*, vol. 10, p. 2365, 2022, DOI: 10.3390/math10142365.
- [27] P. Fischer, A. Dosovitskiy, E. Ilg, P. Häusser, C. Hazirbas, et al., "FlowNet: Learning Optical Flow with Convolutional Networks," *CoRR*, vol. abs/1504.06852, 2015. arXiv: 1504.06852. Available: <http://arxiv.org/abs/1504.06852>.
- [28] J. L. Barron, D. J. Fleet and S. S. Beauchemin, "Performance of optical flow techniques," *International Journal of Computer Vision*, vol. 12, no. 1, pp. 43–77, 1994, DOI: 10.1007/BF01420984. Available: <https://doi.org/10.1007/BF01420984>.
- [29] B. K. P. Horn and B. G. Schunck, "Determining optical flow," *Artificial Intelligence*, vol. 17, no. 1, pp. 185–203, 1981, DOI: 10.1016/0004-3702(81)90024-2.

-
- [30] J. Sánchez, E. Llopis and G. Facciolo, "TV-L1 optical flow estimation," *Image Processing On Line*, vol. 3, pp. 137–150, 2013, DOI: 10.5201/ipol.2013.26.
- [31] G. Farnebäck, "Two-Frame Motion Estimation Based on Polynomial Expansion," 2003, pp. 363–370, ISBN: 978-3-540-40601-3. DOI: 10.1007/3-540-45103-X_50.
- [32] R. Szeliski, "Computer Vision: Algorithms and Applications," 2021.
- [33] H. Moravec, "Obstacle Avoidance and Navigation in the Real World by a Seeing Robot Rover," Pittsburgh, PA, 1980.
- [34] D. Nister, "An efficient solution to the five-point relative pose problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004, DOI: 10.1109/TPAMI.2004.17.
- [35] K. L. Lim and T. Bräunl. "A Review of Visual Odometry Methods and Its Applications for Autonomous Driving," arXiv:2009.09193 [cs]. Sept. 2020. DOI: 10.48550/arXiv.2009.09193.
- [36] OpenCV. "OpenCV: ORB (Oriented FAST and Rotated BRIEF)," Available: https://docs.opencv.org/4.x/d1/d89/tutorial_py_orb.html [visited on 04/27/2025].
- [37] OpenCV. "OpenCV: FAST Algorithm for Corner Detection," Available: https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html [visited on 04/27/2025].
- [38] OpenCV. "OpenCV: BRIEF (Binary Robust Independent Elementary Features)," Available: https://docs.opencv.org/3.4/dc/d7d/tutorial_py_brief.html [visited on 04/27/2025].
- [39] Deep. "Introduction to FAST (Features from Accelerated Segment Test) — deepanshut041," <https://medium.com/@deepanshut041/introduction-to-fast-features-from-accelerated-segment-test-4ed33dde6d65>. [Accessed 29-04-2025].
- [40] E. Marchand, H. Uchiyama and F. Spindler, "Pose Estimation for Augmented Reality: A Hands-On Survey | Request PDF," *ResearchGate*, 2024, DOI: 10.1109/TVCG.2015.2513408. Available: https://www.researchgate.net/publication/287348989_Pose_Estimation_for_Augmented_Reality_A_Hands-On_Survey [visited on 04/29/2025].
- [41] OpenCV. "OpenCV: Perspective-n-Point (PnP) pose computation," 2024. Available: https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html [visited on 04/29/2025].

Appendix

A	Chapter Anhang	xi
A.1	Section Anhang	xi
A.1.1	Subsection Anhang	xi
B	Chapter Anhang 2.....	xv

A Chapter Anhang

A.1 Section Anhang

A.1.1 Subsection Anhang

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

B Chapter Anhang 2



Figure B.1: Bild im Anhang

Table B.1: Tabelle im Anhang

Spalte 1	Spalte 2
...	...